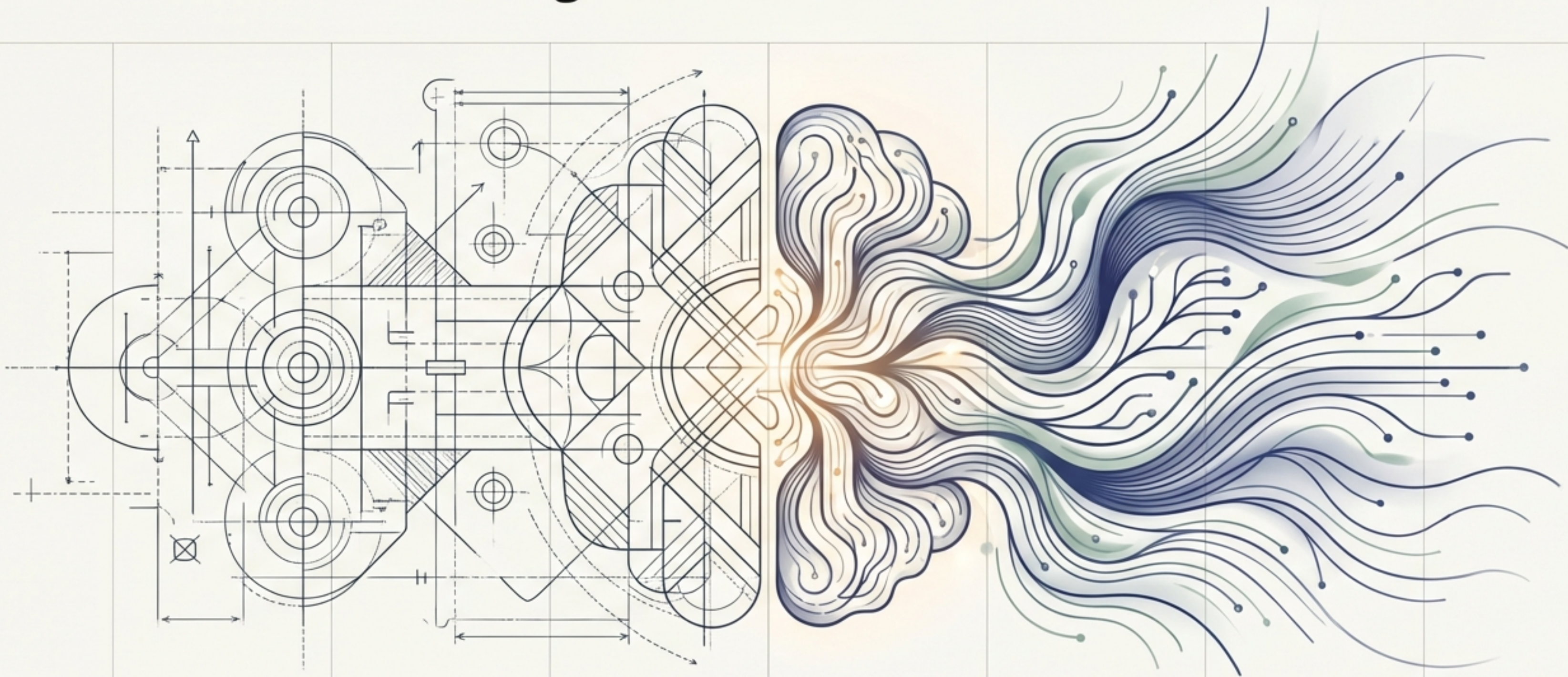


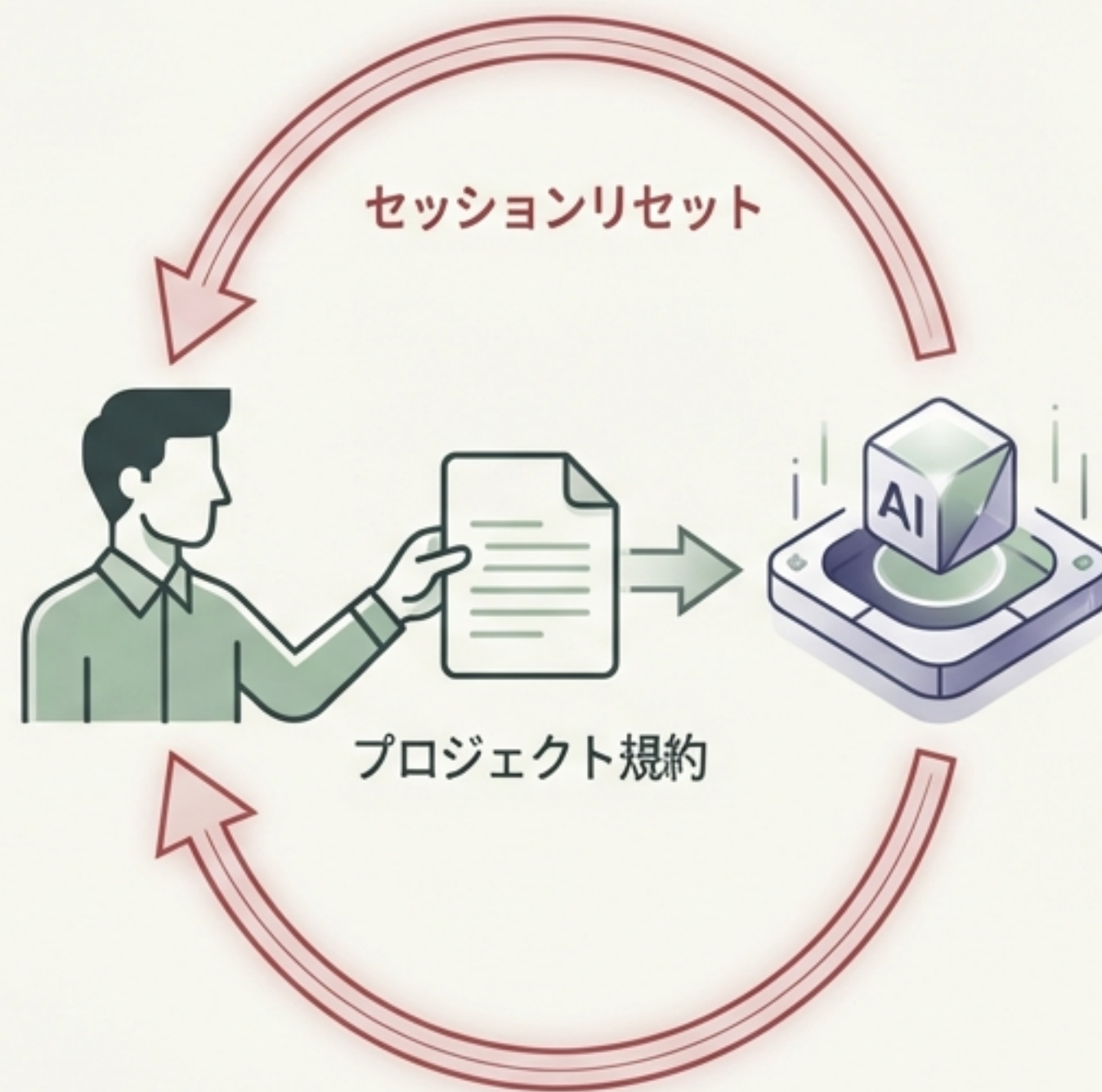
AIエージェントを、単なる「アシスタント」から「エキスパート」へ変える。 Claude Agent Skills 完全マスターガイド



「この指示、前にもしましたよね？」 開発における“デジャブ”の解消

- セッションごとに繰り返される同じ説明とコンテキスト設定
- プロジェクト固有のルールやフォーマットを忘れてしまう
- 品質が安定せず、試行錯誤のコストが積み重なる

Core Problem: LLMはステートレスであり、セッション間で記憶を保持しないアーキテクチャ上の制約がある。



AIに「オンボーディングガイド」
を渡すという発想。

Agent Skillsとは？

新入社員に渡すオンボーディングガイドのように、指示・スクリプト・リソースをパッケージ化した「知識フォルダ」です。

目的

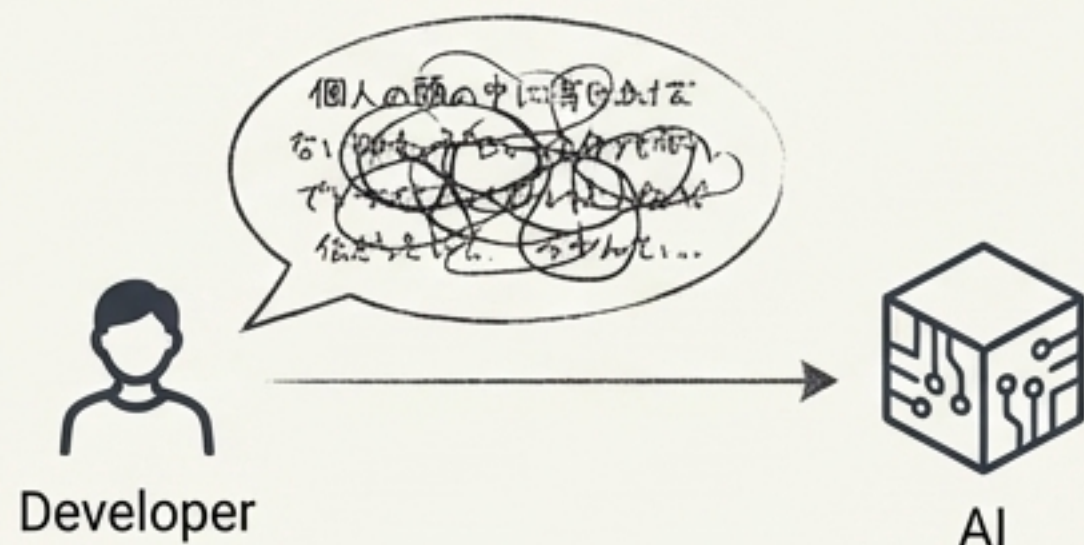
AIの「使い方」を教えるのではなく、
AIの「専門知識」そのものを拡張します。

一度作っておけば、関連するタスクでAIが
自動的にその知識を活用します。



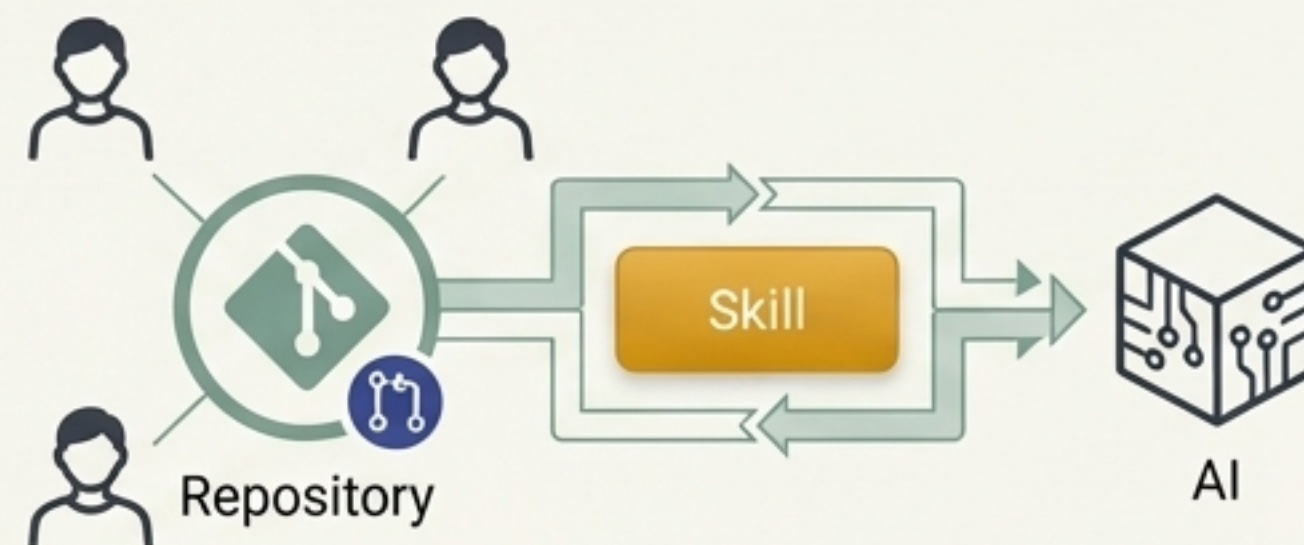
プロンプトエンジニアリングから、ワークフローエンジニアリングへ。

Before: Prompt Engineering



- 個人の頭の中にあった暗黙知を、その都度プロンプトで伝える。
- 属人的で、再現性が低い。

After: Workflow Engineering with Skills



- チームで共有・改善できる「組織の資産」としてワークフローを形式知化する。
- バージョン管理可能で、再利用性が高い。

これからのAIエンジニアリングは、
「LLMの推論をどう制約し、どう組織の資産として蓄積するか」が問われる時代へ。

コア原則: Progressive Disclosure (段階的開示)

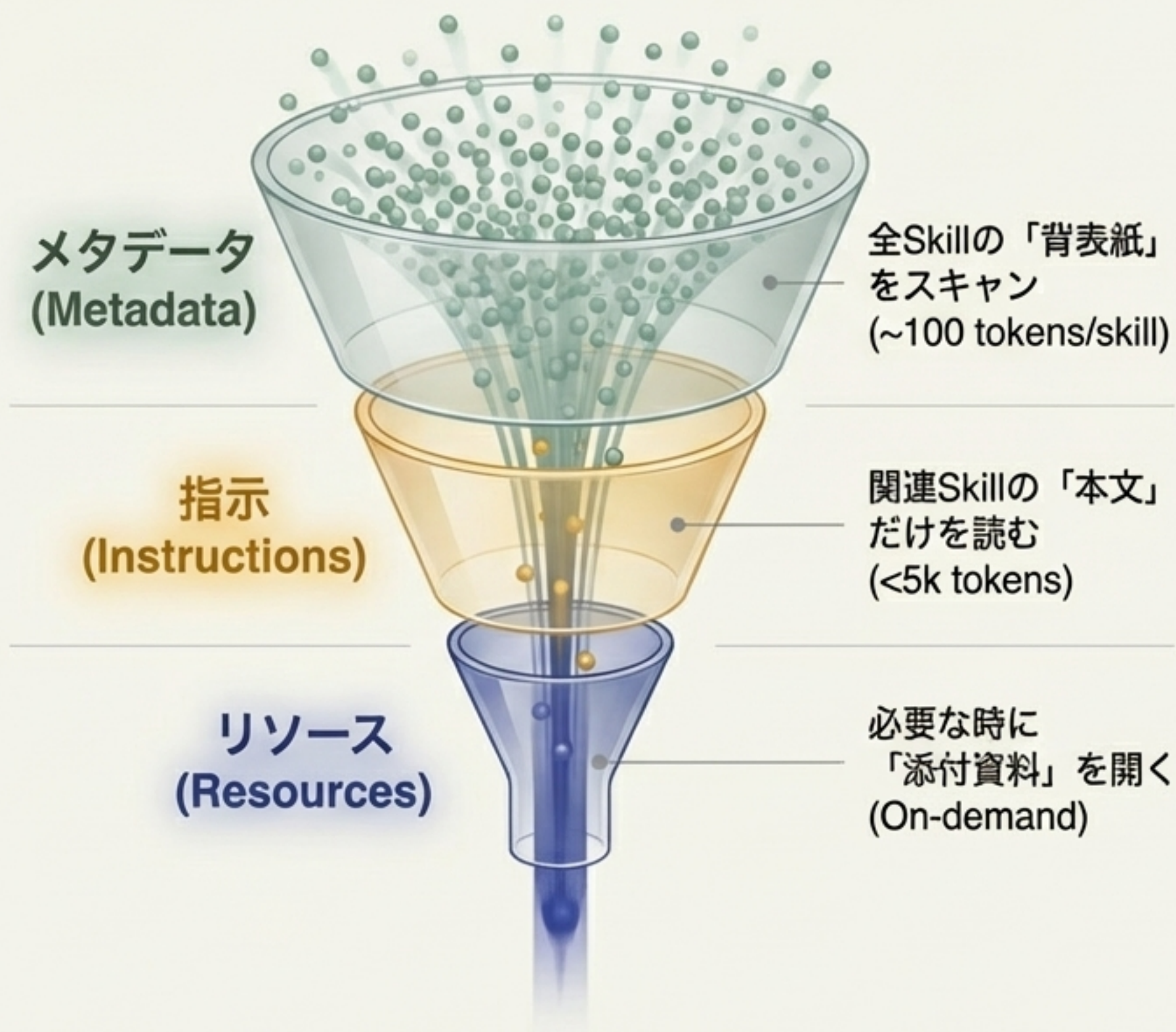
The Problem:

LLMには「コンテキストウィンドウ」という物理的な制約があります。すべての知識を常に読み込むと、すぐに溢れてしまいます。

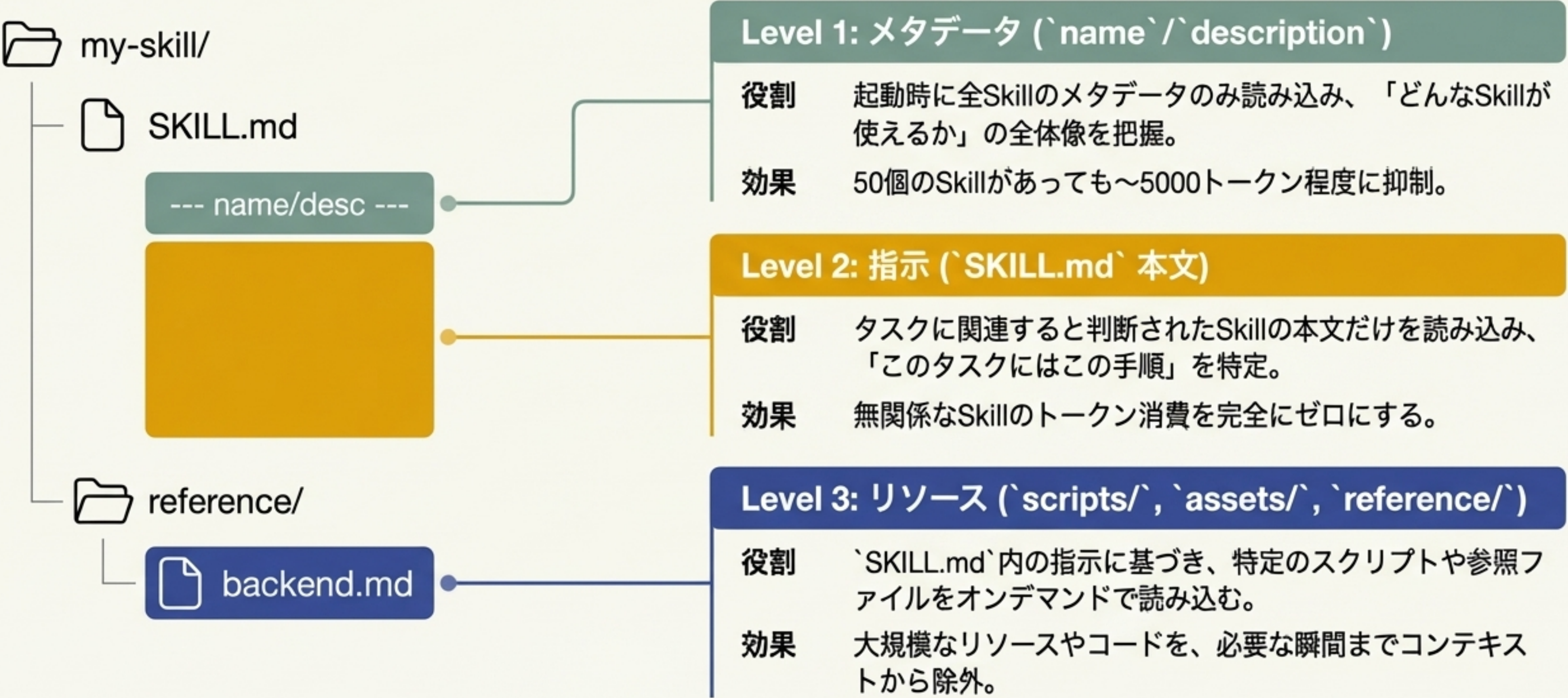
The Principle:

全ての情報を最初から渡すのではなく、必要になったタイミングで必要な情報だけを渡す。

AIの短期記憶（コンテキストウィンドウ）を無駄遣いしないための賢い仕組みです。



推論空間の段階的絞り込み: 3つのレベル



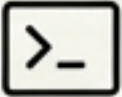
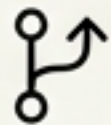
Skillsは「AIエージェントのための振る舞い駆動開発(BDD)」である。

The Analogy: BDDがテストで「コードの振る舞い」を保証するように、SkillsはAIエージェントの「振る舞い」を保証します。

The Benefit: 現場のドメインエキスパートが自然言語で書いた手順が、そのままAIの「生きた仕様書」になります。プログラミングなしで、AIの振る舞いを定義・保証できるのです。

振る舞い駆動開発 (BDD)	Agent Skills
Given (前提条件)	`description` (いつ起動するか)
When (アクション)	`SKILL.md` 本文 (何をするか)
Then (期待結果)	具体的な手順・リソース (どう振る舞うか)

各機能の役割分担: いつ、何を使うべきか

機能	役割	ひと言で言うと
 CLAUDE.md	プロジェクトの全体文脈	「うちはTypeScriptで、こういうアーキテクチャ」
 Commands	手動で呼び出すショートカット	`/test-and-commit` で一連の作業を実行
 Hooks	特定イベントでの自動実行	ファイル保存後に自動フォーマット
 Subagents	独立した専門家の呼び出し	デバッグ時に`debugger` subagentが起動
 Skills	注入可能な専門知識	PDF操作、独自ワークフローのオンボーディング

他の設定がClaudeの「使い方」を定義するのに対し、SkillsはClaudeの「専門知識」そのものを拡張し、「できること」を増やします。

カスタムSkillの基本構造: `SKILL.md`

- Skillの最小構成は `SKILL.md` ファイル1つだけです。
- 配置場所: プロジェクト内なら `.claude/skills/`、全プロジェクト共通なら `~/.claude/skills/`

```
.claude/skills/my-first-skill/SKILL.md
```

```
---
```

```
# 必須フィールド
```

```
name: my-first-skill # 小文字とハイフンのみ
```

```
description: このSkillが何をするか、いつ使うべきかを簡潔に説明。
```

```
---
```

```
# 具体的な指示
```

```
ここにMarkdown 形式で具体的な手順やルールを記述します。
```


実例1: セキュリティレビューの標準化

- **Goal:** 属人化しがちなセキュリティレビューの観点をSkillとして形式知化し、誰が実行しても同じ品質を担保する。
- **Key Takeaway:** `description`に「Use when:」を明記することで、起動トリガーを明確に制御する。

SKILL.md

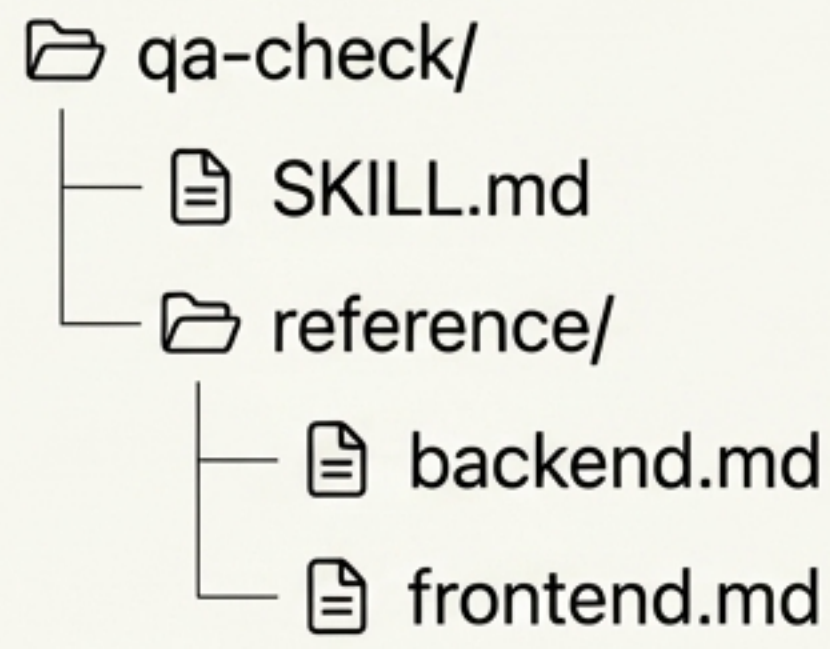
```
---
name: reviewing-security
description: "OWASP API Security Top 10 と Rust セキュリティベストプラクティスに基づき脆弱性を検出。**Use when: セキュリティ、脆弱性、OWASP、認証、認可、監査を依頼された時.**"
---
# セキュリティレビュー

## OWASP チェック項目
| ID      | リスク      | チェック内容      |
| :----- | :----- | :----- |
| API1    | BOLA      | tenant_id 検証、file_id との組み合わせ検証 |
| API2    | Broken Auth | gRPC メタデータ認証 |
| ...     | ...      | ...      |
```


実例2: QAチェックでの段階的開示

- **Goal:** 巨大になりがちなQAチェックリストを、対象領域（例：バックエンド vs フロントエンド）に応じて分割し、コンテキスト消費を最適化する。
- **Key Takeaway:** SKILL.md は目次として機能させ、具体的なチェックリストは外部ファイルにて分離。タスクの文脈に応じて、必要なリファレンスファイルだを読み込ませる。

File Structure



SKILL.md

QAチェック

職種別のリファレンスを参照してレビューを実施します。

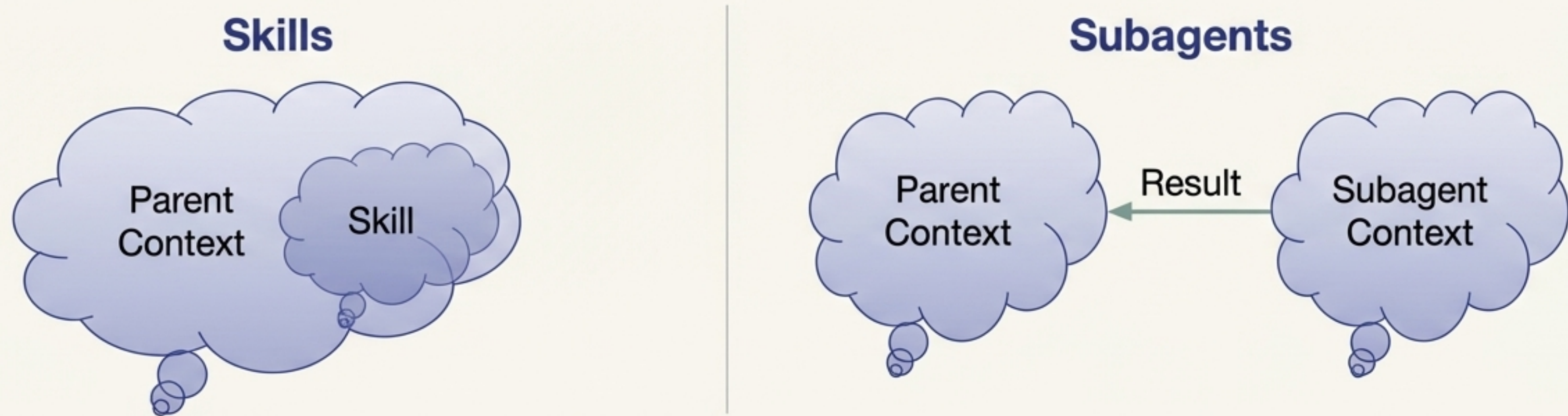
****Rust バックエンド**** → See [[reference/backend.md](#)]

****フロントエンド**** → See [[reference/frontend.md](#)]

高度な使い分け: Skills vs. Subagents

観点 (Aspect)	Skills	Subagents
コンテキスト (Context)	親と共有 (Shared with parent)	独立 (Isolated)
向いているタスク (Suitable Tasks)	継続的な作業、TDDなど対話が重要なタスク	試行錯誤が多い調査タスク、コンテキストを汚したくない作業
状態の引き継ぎ (State Transfer)	あり (Yes, full state)	なし (結果のみ返す)

判断基準: 結果だけでなく「過程」も重要ならSkills。結果の「要約」で十分ならSubagent。



失敗しないためのSkill作成術 ①

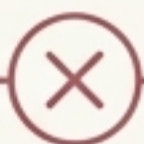
1. 簡潔に書く (Be Concise)

理由: トークン消費とAPIコストを節約し、ノイズを減らすことでLLMの指示理解度を上げる。



Do

```
## Extract PDF text  
Use pdfplumber: ...
```



Don't

PDF (Portable Document Format) files are a common file format...
To extract text from a PDF, you'll need to use a library...

2. 自由度を適切に設定する (Set the Right Level of Freedom)

理由: タスクのリスクと創造性の価値を天秤にかける。
LLMは指示が曖昧だと創造的に解釈する。



高自由度 - 創造的なタスク

```
## Code Review  
1. Analyze structure  
2. Check for bugs...
```



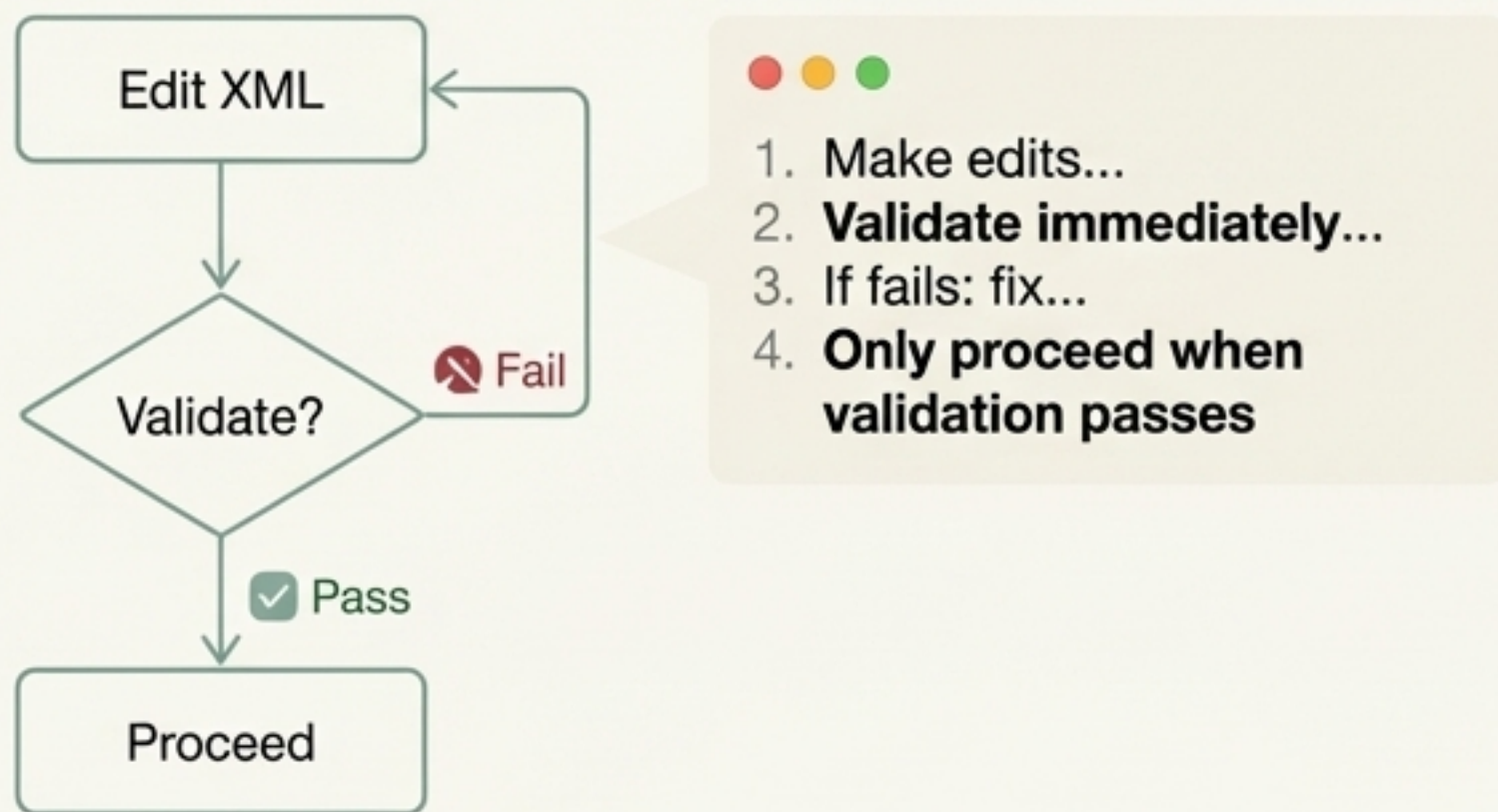
低自由度 - 高リスクなタスク

```
## Database Migration  
Run exactly this  
script: python  
scripts/migrate.py...  
Do not modify.
```


失敗しないためのSkill作成術 ②

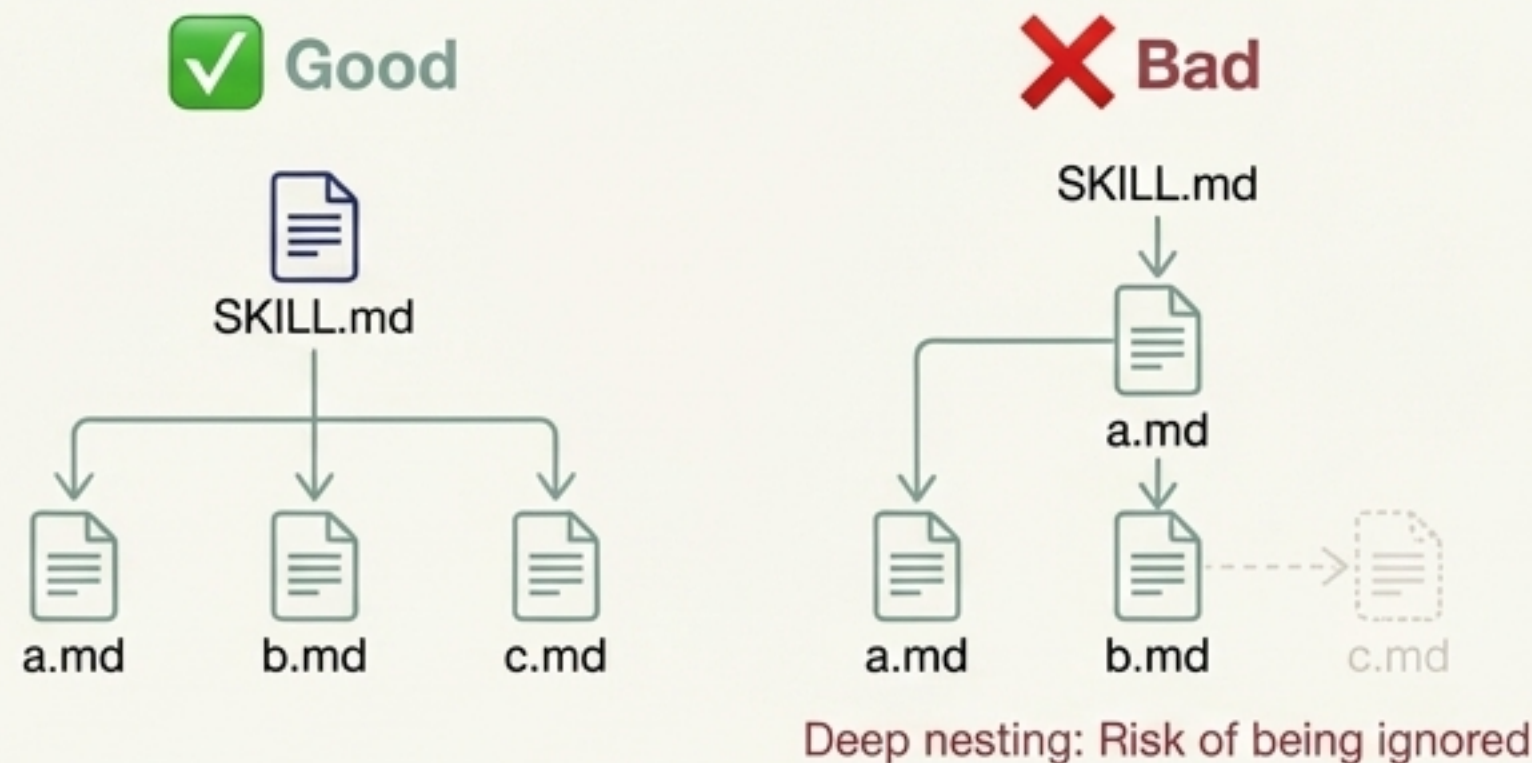
3. フィードバックループを入れる (Include Feedback Loops)

理由: LLMは「確信を持って間違える」ことがある。
複雑なワークフローでは中間検証ステップを挟み、
早期に失敗を検出する。



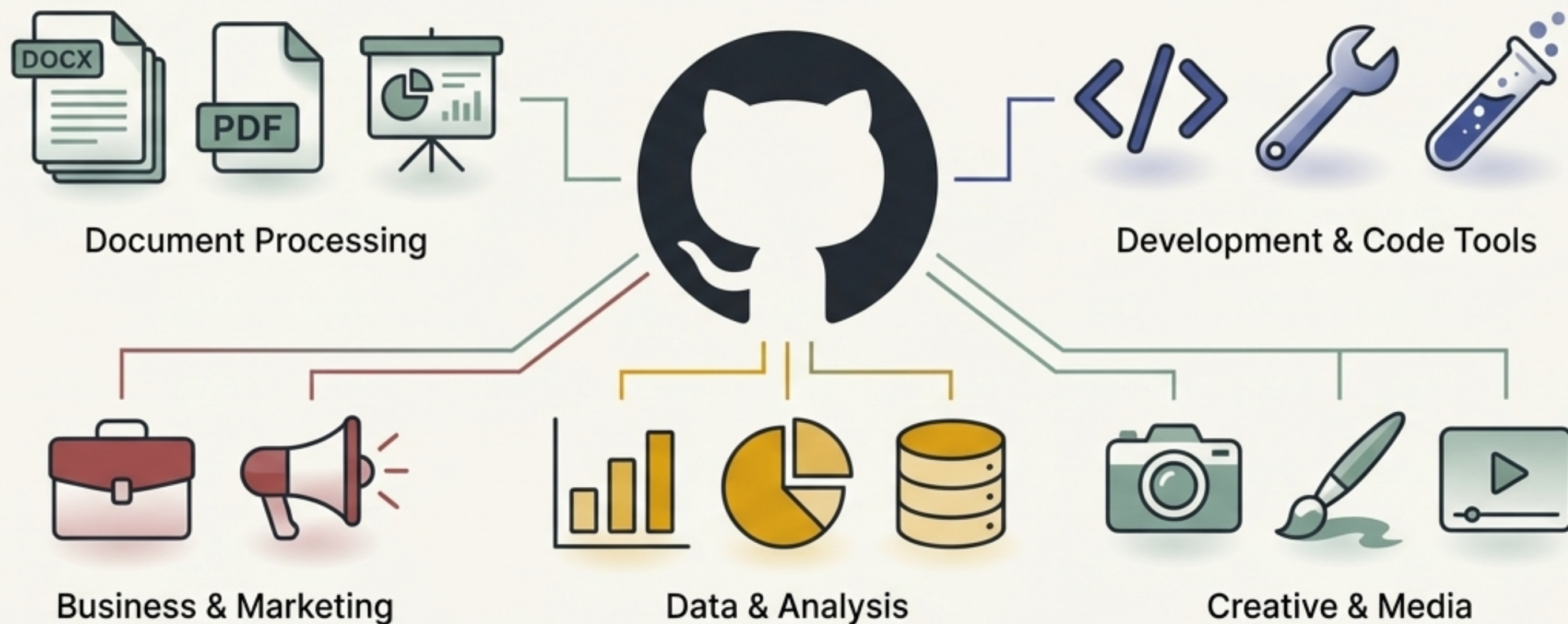
4. ネストを深くしない (Avoid Deep Nesting)

理由: LLMは参照先をどこまで読むかを自分で判断するため、深いネストは無視されるリスクがある。
推奨: 参照ファイルは `SKILL.md` から1階層に留め、
LLMが確実に読み込むようにする。



車輪の再発明は不要: 100以上の公開Skills

`awesome-claude-skills` リポジトリには、すぐに使える実用的なSkillが多数公開されています。コミュニティの作例から、自身のSkill作成のヒントを得ることができます。



[travisvn/awesome-claude-skills](https://github.com/travisvn/awesome-claude-skills)

現実的な課題：Skillsは万能ではない



① 定義ファイル地獄 (Definition File Hell)

Skillが増えると依存関係がブラックボックス化し、管理が困難に。



② `description`の試行錯誤 (Trial-and-Error with `description`)

起動するかはLLMの解釈次第。決定論的なルールではなく、「こう書けば必ず動く」という保証はない。



③ デバッグの難しさ (Difficulty of Debugging)

「なぜこのSkillが起動しなかったのか」というLLMの判断過程が不透明。



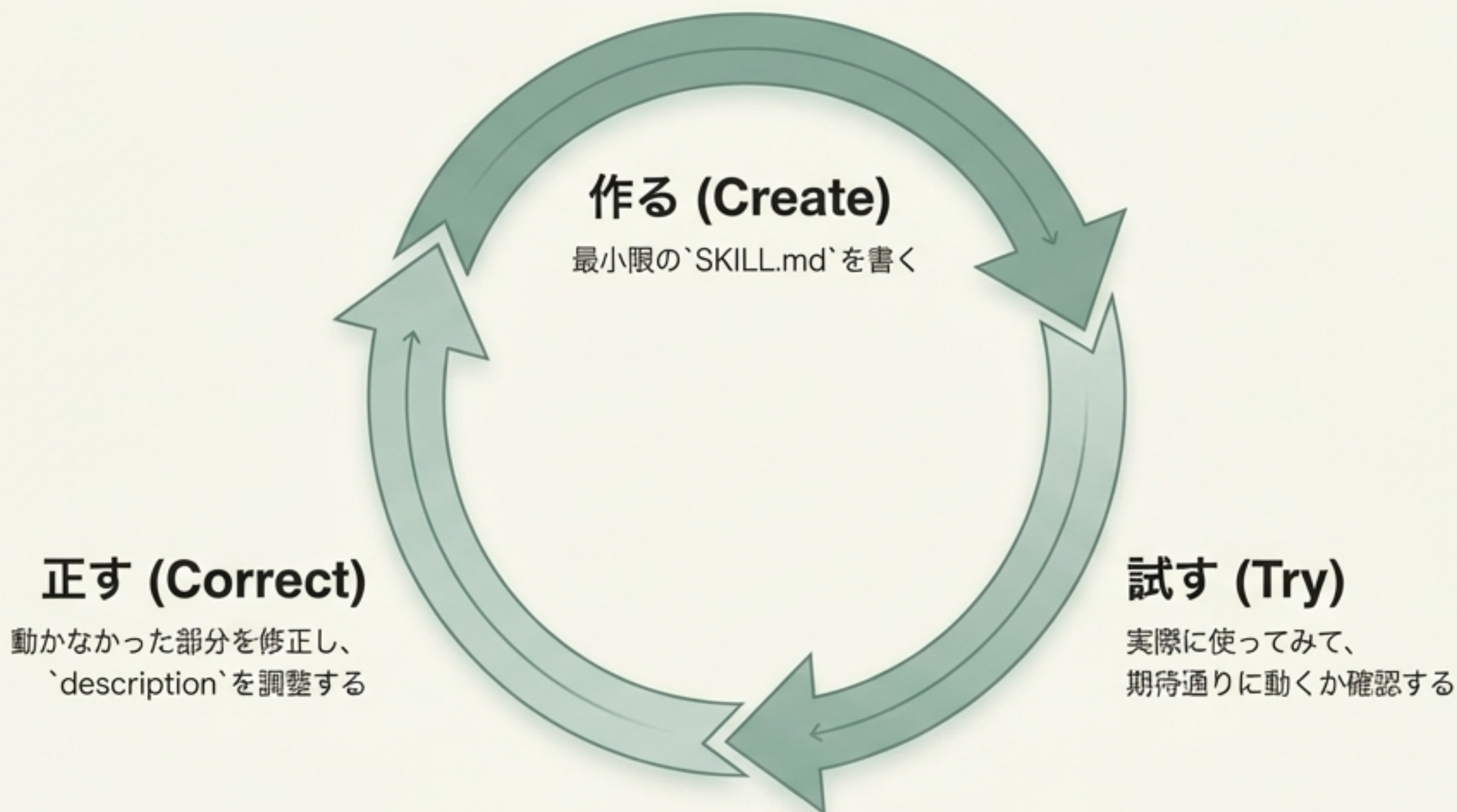
④ Skill同士の競合 (Skill Competition)

似た`description`を持つSkillの選択が予測不能。明示的な優先度設定もない。

「正しさ」を探すな、「正しくなる状況」をつくれ。

The Mindset Shift

Skillの完成度を追い求めるのではなく、適切なタイミングでSkillが発動し、結果としてソフトウェアが正しい方向に進む—その「状況」を設計することが本質です。



Skillは、LLMの振る舞いを理解するための「実験装置」である。

- **Learning Through Experimentation:** **description**の試行錯誤を通じて、私たちはLLMの「**判断基準**」を逆算的に学んでいる。
- **Process Over Product:** Skillsの価値は「**完成品**」ではなく「**育てるプロセス**」にあります。



実践: SkillとHooksを組み合わせた「必須コードレビュー」

The Problem: Skills are "model-invoked," meaning the AI decides whether to use them. There's no guarantee a review skill will run.

The Solution: Separate the "procedure" from the "enforcement."



Skill (`codex-review`)

役割: レビューの「手順」を標準化し、成果物(`review.json`)の置き場所を定義。



Hook (`PreToolUse` on `git commit`)

役割: `git commit`の実行をブロックする「強制力」を持つ。
最新の差分に対する`review.json`が存在しない限り、commitを失敗させる。



CI (GitHub Action)

役割: ローカルのフックを回避された場合の最終的な「品質ゲート」を設ける。



Validated

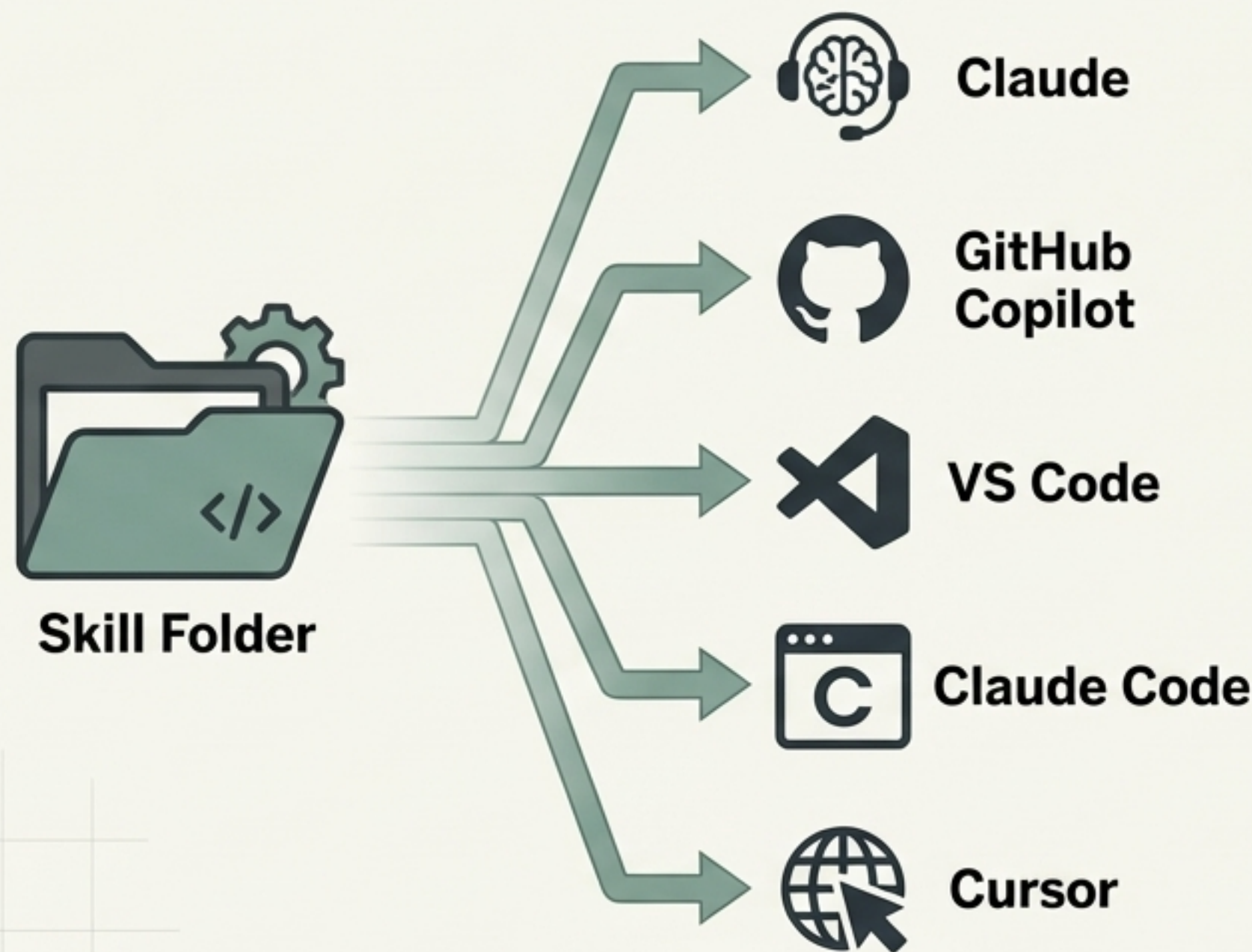
Agent Skillsはオープンスタンダードへ



- Anthropicによって開発され、AIプラットフォームを跨いで動作するオープンな仕様として公開されました。
- 仕様とリファレンスSDKは `agentskills.io` で公開。
- **This means:** AIプラットフォームを跨いで、同じSkillが動作する。

Write Once, Use Everywhere.

なぜオープン標準が重要なのか



- **For Developers:**

- 一度作ったスキルを、好みのツールで再利用できる。学習コストが下がり、生産性が向上します。

- **For Enterprises:**

- 特定ベンダーへのロックインを回避し、手続き的知識をポータブルな資産として蓄積できます。

- **For the Ecosystem:**

- 相互運用性が高まり、より豊かで強力なSkill市場が生まれます。車輪の再発明が減り、イノベーションが加速します。

AI開発の未来像: 知的資産のモジュール化

AIとの協働は、巨大なプロンプトの管理から、モジュール化されたワークフローの組み立てへと進化します。Skillsは、チームの暗黙知を、バージョン管理可能で再利用可能な「組織の知的資産」へと変えます。



今日から始められる3つのステップ



1. 体験する (1分)

Action: `claude.ai`で「Power Pointで自己紹介スライドを作って」と依頼する。

Result: ビルトインSkillが自動で起動し、その力を体験できます。



2. 作ってみる (5分)

Action: 最小構成のカスタムSkill (`hello-skill`) を `~/ .claude/skills/` に作ってみる。

Result: 自分の言葉でAIの振る舞いを定義する感覚を掴めます。



3. 眺めてみる (10分)

Action: `awesome-claude-skills` リポジトリでコミュニティの作例を参考にする。

Result: 実用的なユースケースや高度な書き方のヒントを得られます。